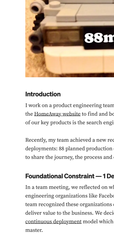

Road to 88 Deployments per Month

Chris Cantu Follow 6 min read · Aug 6, 2018

28 2

Introduction

I work on a product engineering team focused on serving travelers who use the HomeAway website to find and book beautiful vacation properties. One of our key products is the search engine results page (SERP).

Recently, my team achieved a new record for our web application deployments: 88 planned production deployments in a single month. I'd like to share the journey, the process and our learnings.

Foundational Constraint — 1 Deploy per Commit

In a team meeting, we reflected on what some of the most advanced engineering organizations like Facebook, Google and GitHub practice. The team recognized these organizations continuously deploy to production and deliver value to the business. We decided we would follow suit and use a continuous deployment model which included a release per commit to master.

Benefits

The benefits for performing so many releases may not be immediately obvious. Getting to this level of automation took a great deal of investment but it was a decision we firmly believe to have been worth the effort.

Atlassian states their business reasons for continuous deployments:

> Continuous deployment is an excellent way to accelerate the feedback loop with your customers and take pressure off the team as there isn't a Release Day anymore. Developers can focus on building software, and they see their work go live minutes after they've finished working on it.

Beyond Atlassian's observation, we also observed the following benefits.

- ***CICD: Continuous Improvement via Continuous Discipline:*** Our team found what was painful and did it as often as possible. This practice forced us to address pain points in the deployment pipeline and development workflow. The level of engineering and product discipline improved tremendously over time.
- ***Releases are deterministic***: Releasing one thing at a time enables developers to more easily determine the impact of a single change or feature.
- ***Commit history as release history***: Our commits have a 1:1 ratio to our deployments. We can easily track when items are released to production by reviewing commits in GitHub. There is no need for more involved release documentation.
- ***Feature prioritization — no problem***: Changing priorities are an inevitability in a rapid pace work environment. Because features are independent (in their own branches) and easily released the team can easily adjust to new priorities.
- ***Focused delivery***: Stories are not done unless they are in production. Developers remained focused until a feature is in production. Developers are able to finish a story before moving onto other work.
- ***Encourages small stories***: Since deployments are cheap and regular, the team is encouraged to deliver small features. Large features can be delivered in multiple deployments easily.

Top highlight

Branching Strategy

To establish a continuous deployment model, we needed to decide how we will manage code. We initially reviewed the following strategies:

- Git Flow — Includes long living dev, test and production branches; feature branches are branched from dev
- Gitlab Flow — Includes long living production branch; features branches are branched from master
- GitHub Flow — No long living branches. Feature branches are branched from master and deployed to production. Validated changes are merged into master.

We elected to leverage a GitHub Flow like model for its simplicity. To date, we are not deploying from feature branches but rather merging to master and then deploying.

Testing

The next fundamental component of a continuous deployment is our testing strategy. We set out to create a test suite which would give us the confidence to deploy continuously. Our entire test suite is executed on every production release. The team set a 100% unit test coverage as well as 100% pass rate for all production deployments. Pull requests are rejected which do not adhere to our 100% coverage and pass rate.

Our experience with the legacy platform taught us that unreliable or nondeterministic tests were useless tests. If your tests do not provide you consistent and actionable feedback, they are just noise.

Get Chris Cantu's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email Subscribe

☑ Remember me for faster sign in

After months of iteration, we decided on the following testing strategy:

- **Unit tests** → Validates the inputs / outputs of public interfaces.
- **Multicomponent tests** → Validates the interaction between two or more components
- **Mock functional tests**→ Validates the functionality of the UI in a browser with mocked datasources
- **Visual regressions** → Validates CSS changes in a browser by comparing screen captures
- **Smoke tests** → End to End Tests executed in production which are designed to validate the application is usable in production.

Development / Deployment Workflow

Developers begin work by branching off of master. (Master is inherently production given the aforementioned branching strategy.) The process consists of the following:

Pull Requests

Developers create a pull request once code is shareable. Pull requests are scoped to a single user feature or technical deliverable. Our Jenkins integration with GitHub automatically creates a build and deploys successful builds to the test environment. The developer is notified of the status of the build via Slack direct message. Each test deployment URL is based on the JIRA number provided in the branch name.

With the branch-specific deployments, product and design are involved in the pull request process. The team directly requests for approval from design and product for all product or UI features. This innovation ensures we are aligned on the completeness of features before they leave development.

Dev Done

In order for a feature to leave development, it must meet the following criteria:

- Branch up to date: "Master + Feature"
- PR approved by engineering, product and design
- 100% of unit tests passing with 100% coverage
- 100% of mock functional tests pass
- 100% of visual regression tests pass

Deployments

Deployments are largely transparent to developers. Deployments begin automatically once a pull request is merged. Our project integrates with GitHub and our deployment tooling to start a production deployment. Once the deployment is complete, a Slack message is posted in a slack room with the deployment status.

Lessons Learned

Achieving 88 deployments required a great deal of testing and learning over the last 9 months. Some of our key learnings are:

- ***Small iterative steps*** → Implementing a reliable CICD pipeline is a long iterative process. Identify your pain points, prioritize them, and address them one at a time.
- ***Use a ratchet approach to improve test coverage*** → Instead of trying to improve your test coverage all at one time, leverage a library like jest-ratchet to improve your coverage over time.
- ***Deterministic functional automation is hard*** → Look to push your testing logic as close to the code as possible. Multi-component tests are a method to validate functional interactions between components.
- ***Use Saucelabs to perform visual regressions*** → Comparing visual images between two different operating systems is unreliable. Though slower, leverage Saucelabs to compare snapshots.
- ***Deployment speed matters*** → Teams should refrain from adding long-running build steps to their pipeline. Taking tips from another team's work, we recently parallelized some of our build steps, shaving 4 minutes of the process.
- ***Keep WIP small*** → Developers should focus on a single story through production deployments. While their work is in review, they can assist with other code reviews.

Next Steps

The SERP team is still on its journey to becoming a disciplined CICD team. We plan on continued investment in the following areas:

- **Multicomponent testing** → We are prototyping this approach to prevent regressions in our next sprint.
- **Google Lighthouse** → We will integrate Google Lighthouse into our PR process to up our adherence to accessibility and performance best practices.
- **Smoke testing production** → We would like to implement a production smoke test to be run on every deploy. We are currently working on this feature.
- **GitHub Flow** → Our tooling at this time does not support a full GitHub Flow model, though Jeffrey Russom has created a design to enable the process. A true GitHub Flow model is within our grasp and we hope to implement the full approach in the next few sprints.

Continuous Integration Cicd Continuous Deployment Tech

28 2

Published in HomeAway Tech Blog Follow

370 followers · Last published Jul 3, 2019

Software and data science revolutionizing vacations

Written by Chris Cantu Follow

17 followers · 43 following

Responses (2)

Write a response

What are your thoughts?

Brian Tomlin
Aug 10, 2018

I work at HA so you can just slack me :)

Reply

Brian Tomlin
Aug 10, 2018

This is awesome. I'd love to know how you guys are doing mock functional tests. Do you have a repo I could look at?

Reply

More from Chris Cantu and HomeAway Tech Blog

In HomeAway Tech Blog by Chris Cantu

Team Building—It's All About Me

Bonding teams through vulnerability and trust

Dec 5, 2018 143

In HomeAway Tech Blog by Tim Renner

Write Better Python with Hypothesis

When testing code, we at HomeAway always start with unit tests. For a pure function, we…

Sep 24, 2018 296 1

In HomeAway Tech Blog by Himanshu Verma

Simpson's Paradox In A/B Testing

Results of an A/B test on browser variants don't seem to add up

Sep 19, 2018 69 3

In HomeAway Tech Blog by Adam Haines

The Strangler Pattern in Practice

Five Milestones on the Road to the Cloud

Mar 5, 2019 118 3

See all from Chris Cantu See all from HomeAway Tech Blog

Recommended from Medium

In Write A Catalyst by Dr. Patricia Schmidt

As a Neuroscientist, I Quit These 5 Morning Habits That Destroy You...

Most people do #1 within 10 minutes of waking (and it sabotages your entire day)

Jan 14 35K 628

In Activated Thinker by Shane Collins

Why the Smartest People in Tech Are Quietly Panicking Right Now

The water is rising fast, and your free version of ChatGPT is hiding the terrifying,...

Feb 13 11.9K 522

In CodeX by MayhemCode

Why Thousands Are Buying Mac Minis to Escape Issues with Big...

Something strange happened in early 2026. Apple stores started running low on Mac...

Feb 15 4.3K 72

In Generative AI by Adham Khaled

Stanford Just Killed Prompt Engineering With 8 Words (And I...

ChatGPT keeps giving you the same boring response? This new technique unlocks 2×...

Oct 19, 2025 24K 651

In GitBit by John Gruber

Websites Are Dead. Go Here Instead.

I finally did it. I launched a blog. Then I realized the hard truth.

Feb 10 7.6K 301

In Level Up Coding by Teja Kusireddy

Google Is Quietly Dismantling Everything OpenAI Built

The most dangerous failure in Silicon Valley isn’t bankruptcy. It’s becoming the engine...

Feb 17 4.6K 142

See more recommendations